# Diseño y Evaluación de un Sistema de Generación Aumentada por Recuperación para Documentos Internos de una Empresa.

Hans Harley Ccacyahuillca Bejar, Jose Emilio Atauchi Mamani

atauchimamani@gmail.com

**Resumen.** Consultar los documentos internos de una empresa consume tiempo, y recurrir a personas con experiencia para resolver cada duda consume aún más, además de resultar poco práctico cuando las consultas se repiten. Una solución a este problema es la generación aumentada por recuperación (RAG, del inglés Retrieval-Augmented Generation). En este trabajo se diseñó y evaluó un sistema RAG que responde en lenguaje natural sobre 72 documentos internos de una distribuidora eléctrica peruana, ejecutado íntegramente con modelos de lenguaje abiertos. El sistema combina reconocimiento óptico de caracteres (OCR) estructurado (TeleOCR) con limpieza por tipo de bloque, segmentación (chunking) por sección jerárquica, búsqueda híbrida (embeddings densos y BM25 fusionados mediante RRF), un reranker de tipo cross-encoder y un modelo de lenguaje generador. La evaluación se realizó con 170 preguntas de referencia verificadas por el autor, con métricas de recuperación y de generación calificadas por un único LLM juez (gpt-oss-120b). La sección jerárquica obtuvo el mayor Recall@10 (0,959) entre nueve estrategias de chunking. Entre diez modelos generadores, Gemma 4 26B-A4B ofreció el mejor equilibrio: alucina en el 15,3 % de las respuestas, a siete preguntas del menor valor (11,2 %), rechaza correctamente el 97,6 % de las preguntas sin respuesta y responde en 0,82 s, el segundo más rápido.

**Palabras clave:** Generación aumentada por recuperación · Búsqueda híbrida · RAGAS · Alucinación · Modelos de lenguaje abiertos.

## 1. Introducción

En una empresa con un volumen grande de documentos, responder una consulta sobre un documento obliga a buscar entre muchos archivos o a preguntar a quien tiene la experiencia. Los modelos de lenguaje grandes (LLM) responden en lenguaje natural, pero no conocen los documentos internos de la empresa.

Hay dos formas de darles ese conocimiento: el ajuste fino (*fine-tuning*) y la generación aumentada por recuperación (RAG) [5,2]. El ajuste fino exige reentrenar el modelo cada vez que cambian los documentos, mientras que RAG solo actualiza un índice [2]. RAG resuelve tres problemas:

1. **Contexto:** la biblioteca completa no cabe en la ventana de contexto.
2. **Privacidad:** con modelos abiertos, los documentos no salen de los servidores de la empresa.

3. **Actualización:** un documento nuevo se indexa sin reentrenar.

Este trabajo aplica RAG con modelos abiertos en un entorno empresarial y compara sus componentes por separado.

El caso de estudio son 72 documentos internos de una distribuidora eléctrica peruana. Las contribuciones son: (i) un sistema RAG con modelos abiertos, descrito por fases con la entrada y salida de cada paso; (ii) la comparación de nueve estrategias de *chunking* y de diez LLM generadores sobre las mismas preguntas; y (iii) una evaluación con métricas estándar de recuperación y de alucinación.

## 2. Fundamentos técnicos

*OCR.* Convierte el PDF en texto. Los motores basados en modelos de visión y lenguaje entregan además la estructura del documento (tipo de bloque y página); OmniDocBench [9] los compara en textos, tablas y orden de lectura.

Chunking. Divide cada documento en fragmentos (*chunks*) que se indexan y recuperan como unidad, para no llenar el contexto del LLM. Las estrategias van desde ventanas de tamaño fijo hasta cortes semánticos, por LLM o por la estructura del propio documento [11,3].

*Embedding (búsqueda densa).* Un codificador transforma un texto en un vector que representa su significado; textos parecidos quedan cerca. Se usa un *bi-encoder*: pregunta y chunk se codifican por separado, y los chunks se vectorizan una sola vez al indexar. MTEB [7] compara estos modelos.

*BM25 (búsqueda léxica).* Puntúa por coincidencia exacta de términos según su frecuencia en el chunk y la longitud del chunk; en su forma clásica pondera además cada término por su rareza en la colección (IDF) [10]. Complementa al embedding con códigos y siglas (p. ej. “CVA-PR-004”).

*RRF.* La fusión por rango recíproco [4] combina las listas densa y léxica: cada documento $d$ suma $\mathrm{RRF}(d) = \sum_r 1/(k + \mathrm{rango}_r(d))$ sobre las listas $r$ donde aparece, con $k$ constante. No necesita normalizar puntajes.

Reranker. Reordena los candidatos con un modelo más preciso y más lento. Hay tres arquitecturas: el *bi-encoder* (codificación separada, rápida, la de la búsqueda), el *cross-encoder* (lee pregunta y chunk juntos y emite un puntaje de relevancia) y ColBERT, intermedio, que compara los vectores de cada token. Aquí se usa un *cross-encoder*.

*Base de datos vectorial.* Guarda cada chunk con sus vectores denso y disperso y sus metadatos, y resuelve la búsqueda por similitud.

*Normalización de la consulta.* Antes de buscar, un LLM corrige la ortografía y reescribe la pregunta en un lenguaje más cercano al de los documentos (*query rewriting* [6]).

*LLM generador.* Recibe la pregunta y los chunks recuperados en un *prompt* y redacta la respuesta usando solo ese contexto. Cuando afirma algo que no está en el contexto, *alucina*.

## 3. Arquitectura del sistema

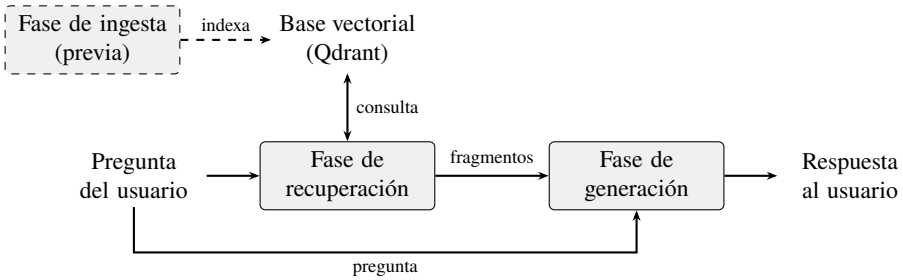



**Figura 1.** Arquitectura del RAG. La ingesta se ejecuta una vez y indexa la base vectorial; en cada consulta, la recuperación obtiene los fragmentos y la generación produce la respuesta.

La Fig. 1 muestra la arquitectura del sistema. A continuación describiremos cada componente de este.

El sistema sigue el flujo clásico de ingesta, indexación, recuperación y generación [2]: la ingesta construye el índice una vez; la recuperación y la generación se ejecutan con cada pregunta. La Tabla 1 resume los componentes elegidos.

**Tabla 1.** Componentes del sistema.

| Componente | Elección | Motivo |
|---|---|---|
| OCR | TeleOCR | Salida estructurada por bloque, reconocimiento de tablas y 1.er lugar en OmniDocBench v1.6 según sus autores [12] |
| Chunking | Sección jerárquica | Mayor Recall@10 y nDCG@10 entre nueve estrategias (Sec. 5.1) |
| Embedding | harrier-oss-v1-0.6b | Abierto (Microsoft), 596 M parámetros, 1 024 dimensiones; buen puesto en MTEB [7] y baja latencia |
| Léxico | BM25 (FastEmbed) | Coincidencia de códigos y siglas; sin factor IDF de colección |
| Base vectorial | Qdrant | Código abierto; búsqueda densa y dispersa con RRF en una consulta |
| Normalización | Qwen3.5-4B | Corrección ortográfica y reformulación de la pregunta |
| Reranker | Qwen3-Reranker-8B | Cross-encoder; reordena los 10 candidatos de la búsqueda híbrida |
| Generador | Gemma 4 26B-A4B (FP8) | Mejor equilibrio entre alucinación, rechazo correcto y latencia; MoE con ∼3,8 B de 25,8 B parámetros activos por token (Sec. 5.2) |

### 3.1. Ingesta

La Fig. 2 muestra la entrada y salida de cada paso. TeleOCR [12] se eligió porque, además del Markdown, entrega un JSON en el que cada bloque trae su tipo (título, texto,

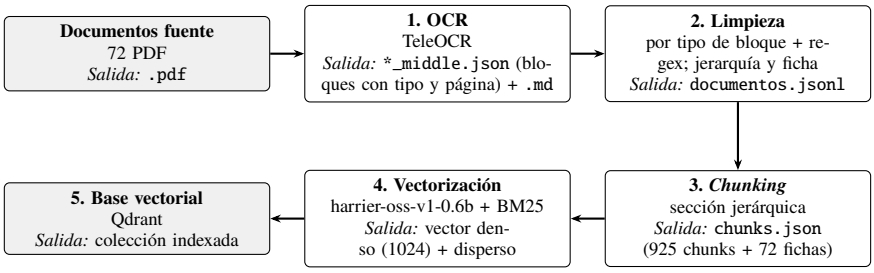



**Figura 2.** Fase de ingesta: de los PDF a la colección en Qdrant. Se ejecuta una vez, o cuando cambian los documentos.

tabla, encabezado, pie) y su página. Eso permite limpiar *por tipo de bloque* y no solo con expresiones regulares: se eliminan pies, números de página y encabezados repetidos en dos o más páginas; con expresiones regulares se quita el membrete de cada página ("Código / Versión / Página N de M") y los bucles de texto repetido del OCR; las tablas HTML pasan a Markdown. La limpieza reconstruye también la jerarquía de títulos a partir de la numeración ("2.", "2.1") y extrae la ficha de aprobación de cada documento, que luego se indexa como un chunk propio.

El *chunking* por sección jerárquica crea un *chunk* por título numerado (925 chunks, mediana de 283 tokens), sin usar modelos; con la ficha de cada documento se indexan 997 chunks. A cada chunk se le antepone el nombre del documento y la sección antes de vectorizarlo, y se guarda con su vector denso y su vector disperso BM25 en Qdrant. El vector BM25 se calcula con FastEmbed y la colección no aplica el factor IDF, por lo que la búsqueda léxica pondera solo la frecuencia de cada término en el chunk.

### 3.2. Recuperación

La Fig. 3 muestra el flujo. La pregunta se corrige y se reformula con un LLM pequeño; luego se buscan en paralelo los 20 candidatos más cercanos por embedding y los 20 mejores por BM25, se fusionan con RRF en una lista de 10 y el *reranker* conserva los 5 mejores. La pregunta corregida y reformulada solo se usa para buscar; no pasa al generador.

### 3.3. Generación

Los 5 chunks se insertan en el mensaje de usuario, cada uno con la cabecera `[Fuente i: documento, página, sección]`, seguidos de la pregunta original del usuario, la misma del inicio y sin corregir ni reformular (Fig. 4). El *prompt* de sistema obliga a responder solo con el contexto y, si no encuentra la respuesta, a decir "No encontré esta información en los procedimientos disponibles". El generador responde con temperatura 0,2 y un máximo de 800 tokens. La respuesta incluye las referencias para que el usuario verifique la fuente.

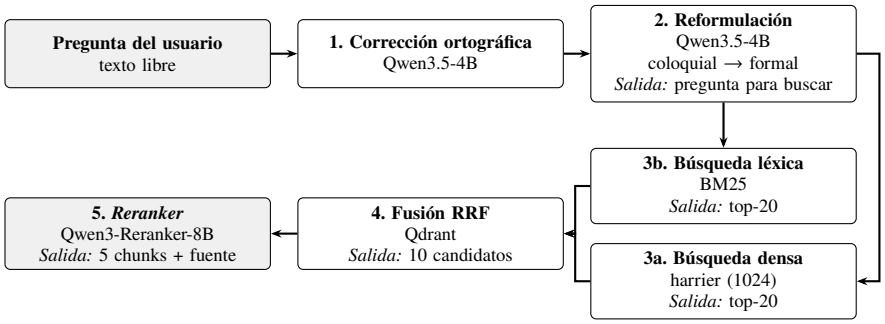



**Figura 3.** Fase de recuperación: de la pregunta a los 5 chunks que recibe el generador. La pregunta corregida y reformulada solo se usa aquí, para buscar; el generador recibe la original (Fig. 4).

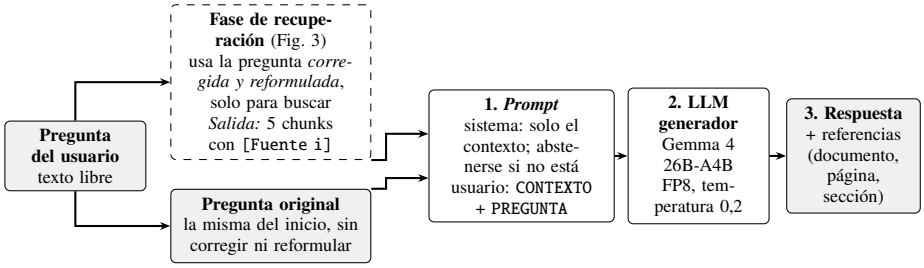



**Figura 4.** Fase de generación. La pregunta corregida y reformulada solo se usa para recuperar los chunks; el *prompt* del generador recibe la pregunta original del usuario, sin cambios, junto con esos chunks.

## 4. Metodología de evaluación

### 4.1. Preguntas de referencia (*golden set*)

Todas las pruebas usan un único conjunto de preguntas de referencia, el *golden set* v3, generado a partir del texto limpio de los 72 documentos (no de los chunks) para que no dependa de ninguna estrategia de chunking. Con una semilla fija se eligieron 194 secciones, unas tres por documento y de ramas distintas de su jerarquía, descartando la portada y las secciones de menos de 60 palabras. Se sumaron las *fichas de aprobación* de 18 documentos elegidos al azar. La ficha es el cuadro de la primera página que indica quién elaboró, revisó y aprobó el documento, con su cargo y fecha; la limpieza la extrae y se indexa como un chunk propio. Sobre cada ficha se hizo una pregunta ("¿quién aprueba. . . ?"), una consulta frecuente en la práctica. En total se partió de 212 textos.

Cada texto pasó por tres pasos. Primero, Llama 3.3 70B escribió una pregunta del tipo sugerido por la sección y copió la *evidencia*, es decir, la oración literal que la responde. Después, el mismo modelo redactó la respuesta de referencia usando solo esa sección. Por último, cada pregunta se validó de dos formas. gpt-oss-120b [8] revisó cinco criterios: que se entienda sola y no sea ambigua entre documentos, que la sección baste para responderla, que la respuesta sea fiel a la sección, que suene a una pregunta

real de un trabajador y que la respuesta sea completa. Además, el código comprobó, sin intervención del juez, que la evidencia aparezca literalmente en la sección o que al menos el 90 % de sus palabras estén en ella.

De las 212 preguntas generadas se descartaron primero 6 casi idénticas a otras. Luego se rechazaron 30 que fallaron la comprobación de la evidencia o alguno de los tres primeros criterios del juez, y se dejaron fuera 5 dudosas que fallaron solo la naturalidad o la completitud. Quedaron 171 aceptadas.

El autor revisó a mano las 171 contra el PDF original y validó 170; la restante se marcó para corregir y se excluye. Todas las pruebas usan estas 170 preguntas verificadas, de 68 documentos, de las cuales 15 son sobre fichas de aprobación.

Cada pregunta tiene un tipo. El título de la sección sugiere uno (las de *Definiciones*, definición; las de *Responsabilidades*, responsable; en las demás se elige al azar entre procedimiento, condicional y dato puntual), y el modelo lo confirma al escribir la pregunta, o usa dato puntual si la sección no permite el tipo sugerido. Los tipos cubren las consultas habituales sobre un documento y ponen a prueba al sistema de formas distintas (Tabla 2).

**Tabla 2.** Tipos de pregunta del *golden set* (170 preguntas verificadas).

| Tipo | *n* | Qué pone a prueba | Ejemplo |
|---|---|---|---|
| Dato puntual | 58 | Encontrar un valor concreto (un código, un plazo, una norma, un sistema) que suele estar en una sola frase del documento | ¿Qué ley regula las concesiones eléctricas en el procedimiento de recaudación de cobranzas? |
| Responsable | 38 | Identificar quién elabora, aprueba o ejecuta algo; se apoya en la ficha de aprobación y en la sección de responsabilidades | ¿Quién aprobó el documento de Impresión de recibos? |
| Definición | 36 | Recuperar el significado de un término en el documento correcto, ya que muchos términos se definen en varios documentos | ¿Qué significa Tolerancia en el contexto del mantenimiento y verificación de equipos de medición? |
| Procedimiento | 24 | Responder una secuencia de pasos completa; exige más contexto y que el *chunking* no separe los pasos | ¿Cuáles son los pasos para ingresar el detalle del motivo de re-facturación en el Sistema Comercial, según el procedimiento de refacturación? |
| Condicional | 14 | Reconocer una situación y la acción que corresponde ante ella | ¿Qué se debe hacer si un cliente presenta una solicitud o reclamo respecto a su consumo de energía eléctrica en el procedimiento de inspección de suministros? |

### 4.2. Prueba de recuperación

La prueba mide si el chunk que contiene la evidencia llega a los primeros puestos y en qué posición. Un chunk es *relevante* si pertenece al documento esperado y su posición en el texto limpio se solapa con la de la evidencia; la ficha de aprobación es relevante si la evidencia está en ella. La evidencia se ubica por búsqueda literal; si la cita resume o une frases y no aparece tal cual (14 preguntas), se toma la zona del texto que reúne al menos el 40 % de sus secuencias de tres palabras. Si la evidencia aparece varias veces en el documento, cualquiera de esas posiciones cuenta como relevante.

Todas las preguntas tienen un documento y una sección de origen conocidos. Lo que esta prueba necesita, además, es ubicar la evidencia dentro del texto, y en cuatro

preguntas (todas de dato puntual) no fue posible. En tres, la evidencia es una sola palabra, "SIELSE" (el sistema que soporta una actividad): es demasiado corta para ubicarla y además aparece decenas de veces en el documento, por lo que no señala ninguna sección. En la cuarta, la evidencia une en una línea dos requisitos que en el texto aparecen como ítems separados de una lista, y no se encuentra ni literal ni de forma aproximada. Esas cuatro preguntas se excluyen solo de esta prueba, que usa 166. La prueba de alucinación sí usa las 170, porque allí el juez compara la respuesta con los chunks y no necesita la posición de la evidencia.

Las métricas se calculan sobre la lista de 10 chunks que deja el *reranker*:

- **Acc@*k*** (*accuracy*): porcentaje de preguntas con al menos un chunk relevante entre los $k$ primeros; Acc@1 indica que el relevante está primero y Acc@5, que llega entre los 5 que recibe el generador.
- **Recall@10**: fracción de todos los chunks relevantes de la colección que aparece entre los 10 primeros.
- **MRR@10**: promedio del inverso de la posición del primer chunk relevante (1 si está primero, 0,5 si está segundo, etc.; 0 si no aparece entre los 10).
- **nDCG@10**: calidad del orden completo; es mayor cuando los chunks relevantes están más arriba.
- **Tokens**: promedio de tokens de los 5 primeros chunks, es decir, la cantidad de texto que leería el generador.

### 4.3. Comparación de estrategias de *chunking*

Para elegir cómo dividir los documentos se compararon nueve estrategias de *chunking* sobre el mismo texto limpio de TeleOCR. Las estrategias representan seis familias de métodos (Tabla 3): por estructura del documento, por tamaño fijo, por similitud entre embeddings, por cohesión léxica, por grafos y con un LLM. Las dos estrategias con LLM (LumberChunker y Dense X) usaron Qwen3.5-9B, y las basadas en embeddings, harrier-oss-v1-0.6b, el mismo modelo de la búsqueda densa.

Todas las estrategias comparten la misma base y solo difieren en el lugar donde cortan el texto:

- La unidad mínima, que nunca se divide, es una oración, un ítem de lista, un título o una tabla; las tablas de más de 400 tokens se dividen por filas.
- Salvo en Fixed Size y Dense X, los chunks de menos de 50 tokens se unen a un chunk vecino, de preferencia de la misma sección, y los de más de 1 000 se dividen.
- El texto que se convierte en vector lleva al inicio el nombre del documento y de la sección ("Documento: . . . | Sección: . . . "), para que el chunk conserve su contexto.
- El primer chunk de cada documento es siempre su ficha de aprobación, que se suma a los chunks de la Tabla 3 (72 más por estrategia).

Cada estrategia se indexó en una colección propia de la base de datos vectorial, con el mismo modelo de embeddings y el mismo índice BM25. Las 166 preguntas de la prueba de recuperación se buscaron en cada colección con el mismo flujo: búsqueda híbrida (20 candidatos por vía), fusión RRF en los 10 primeros y reordenamiento de esos 10 con el *reranker*. Las preguntas se usaron tal como están en el *golden set*, sin corrector ni reformulador, para que la única diferencia entre colecciones fuera el *chunking*. En esta prueba no interviene ningún LLM generador ni juez.

**Tabla 3.** Estrategias de *chunking* comparadas. Chunks y mediana de tokens sin contar la ficha de aprobación de cada documento (72 chunks más por estrategia).

| Estrategia | Familia | Dónde corta | Chunks | Tokens (mediana) |
|---|---|---|---|---|
| Sección jerárquica | Estructural | Un chunk por cada título numerado del documento (1., 1.1, 1.1.1); no usa ningún modelo | 925 | 283 |
| Fixed Size | Tamaño fijo | Ventanas de 512 tokens con 50 de solapamiento, sin mirar el contenido | 822 | 512 |
| Recursive Semantic | Embeddings | Donde la distancia entre oraciones consecutivas es mayor que el percentil 95 | 1 037 | 335 |
| Sequential HAC | Embeddings | Agrupa oraciones contiguas mientras sean similares | 1 122 | 177 |
| Max-Min | Embeddings | Agrega oraciones mientras su similitud con el chunk supere un umbral que se adapta | 1 370 | 141 |
| TextTiling | Léxica | En los puntos donde baja la coincidencia de palabras entre bloques vecinos | 977 | 284 |
| GraphSeg | Grafo | Según los grupos de oraciones muy similares entre sí | 1 238 | 175 |
| LumberChunker | LLM | Un LLM lee el texto y elige la oración donde cambia el tema | 1 255 | 281 |
| Dense X | LLM | Un LLM reescribe el texto como proposiciones breves e independientes | 11 533 | 32 |

### 4.4. Prueba de alucinación

La prueba compara diez modelos generadores dejando fija la recuperación, de modo que la única diferencia entre ellos es el generador. Se realiza en cinco pasos:

1. **Recuperación.** Cada una de las 170 preguntas pasa una sola vez por el flujo de producción: corrector y reformulador (Qwen3.5-4B), búsqueda híbrida BM25 + embedding denso (harrier-oss-v1-0.6b) con 20 candidatos por vía, fusión de ambas listas en los 10 primeros, reordenamiento con Qwen3-Reranker-8B y selección de los 5 primeros chunks, sobre la colección de sección jerárquica. Estos chunks se guardan y se reutilizan para todos los modelos.
2. **Construcción de las dos variantes.** Cada pregunta se presenta dos veces:
   - *Respondible*: con los 5 chunks recuperados para ella. Lo correcto es responder usando solo esos chunks.
   - *No respondible*: con los 5 chunks recuperados para otra pregunta de la misma área de negocio, cuyo documento esperado es distinto y cuyos chunks no incluyen el documento esperado ni la evidencia. Lo correcto es abstenerse con la frase "No encontré esta información en los procedimientos disponibles", como en la prueba de *negative rejection* de RGB [1].

   Resultan $170 \times 2 = 340$ *prompts* por modelo, idénticos para los diez.
3. **Generación.** Cada modelo responde los 340 *prompts* con el mismo *prompt* de sistema de producción, temperatura 0 y semilla fija (la respuesta es siempre la misma), un máximo de 800 tokens y el razonamiento desactivado.
4. **Detección de abstenciones.** Sin intervención de ningún juez, se identifica qué respuestas contienen la frase de abstención.
5. **Calificación con el juez.** gpt-oss-120b, único juez de este trabajo, califica con RAGAS todas las respuestas que afirman algo: las respondibles y las no respondibles en que el modelo no se abstuvo. El juez ve la pregunta, la respuesta, los mismos chunks que vio el generador con su cabecera `[Fuente i: documento, página, sección]` y la respuesta de referencia.

Las métricas obtenidas son:

- **Tasa de alucinación**: porcentaje de las 170 respondibles en que el juez encuentra al menos una afirmación que no está en los chunks (*faithfulness* < 1). Una abstención cuenta como respuesta sin alucinación, de modo que el denominador es 170 para todos los modelos.
- ***Faithfulness***: fracción de las afirmaciones de la respuesta respaldadas por los chunks (de 0 a 1).
- **Rechazo correcto**: porcentaje de las 170 no respondibles en que el modelo se abstiene. No depende del juez.
- **Inventa sin información**: porcentaje de las 170 no respondibles en que el modelo responde y el juez encuentra afirmaciones sin respaldo.
- **Abstención falsa**: porcentaje de las 170 respondibles en que el modelo se abstiene teniendo la respuesta. No depende del juez.
- **Exactitud** (*answer accuracy*): grado en que la respuesta coincide con la de referencia, de 0 a 1.
- **Completitud**: fracción de las afirmaciones de la respuesta de referencia que aparecen en la respuesta.
- **Latencia**: tiempo mediano de respuesta de 20 preguntas respondibles enviadas de a una, con un solo usuario y un modelo por GPU L40S.

La *faithfulness*, la exactitud y la completitud se promedian sobre las respondibles en que el modelo no se abstuvo, porque una abstención no tiene afirmaciones que calificar. Todos los modelos se evalúan sobre la misma base: las 170 preguntas verificadas, cada una en sus dos variantes. Los resultados se presentan en la Sec. 5.2 (Tabla 5).

Las tasas de alucinación deben leerse en términos relativos: el juez marca como no respaldadas algunas afirmaciones correctas, como la atribución final “según el procedimiento X”, lo que eleva por igual las tasas de todos los modelos. Por eso conviene leerlas junto con el rechazo correcto y la abstención falsa, que se miden sin juez.

## 5. Resultados

Todos los resultados usan las preguntas del *golden set* v3 verificadas por el autor (Sec. 4): la prueba de alucinación usa las 170 y la de *chunking*, las 166 cuya evidencia se pudo ubicar en el texto.

### 5.1. Estrategia de *chunking*

La Tabla 4 muestra las nueve estrategias evaluadas con las 166 preguntas de la prueba de recuperación (Sec. 4.3).

Las diferencias entre las estrategias que conservan oraciones completas son pequeñas. Sección jerárquica, Fixed Size, LumberChunker, Recursive Semantic, Sequential HAC y TextTiling quedan entre 0,922 y 0,946 de Acc@5: con cualquiera de ellas, el chunk con la respuesta llega entre los 5 que recibe el generador en más de nueve de cada diez preguntas. Max-Min y GraphSeg quedan por debajo (0,867 y 0,873).

Se eligió la sección jerárquica por cuatro razones:

**Tabla 4.** Resultados de las estrategias de *chunking* (166 preguntas). Tokens: promedio de los 5 primeros chunks. En negrita, el mejor valor de cada columna (en tokens, el menor).

| Estrategia | Familia | Acc@1 | Acc@5 | Recall@10 | MRR@10 | nDCG@10 | Tokens |
|---|---|---|---|---|---|---|---|
| **Sección jerárquica** | Estructural | 0,633 | 0,940 | **0,959** | 0,763 | **0,796** | 1 894 |
| Fixed Size | Tamaño fijo | 0,614 | **0,946** | 0,956 | 0,757 | 0,794 | 2 394 |
| LumberChunker | LLM | 0,657 | 0,922 | 0,949 | 0,762 | 0,792 | 1 567 |
| Recursive Semantic | Embeddings | 0,554 | 0,928 | 0,948 | 0,716 | 0,765 | 1 687 |
| Sequential HAC | Embeddings | 0,614 | 0,922 | 0,945 | 0,741 | 0,773 | 1 737 |
| TextTiling | Léxica | 0,596 | 0,922 | 0,947 | 0,741 | 0,781 | 1 667 |
| Max-Min | Embeddings | 0,602 | 0,867 | 0,938 | 0,722 | 0,762 | 1 448 |
| GraphSeg | Grafo | 0,572 | 0,873 | 0,923 | 0,693 | 0,740 | 1 550 |
| Dense X | LLM | **0,723** | 0,916 | 0,350 | **0,809** | 0,494 | **244** |

1. **Es la que mejor recupera y ordena.** Tiene el mayor Recall@10 (0,959) y el mayor nDCG@10 (0,796). En Acc@5 queda segunda (0,940), a una pregunta de Fixed Size (0,946).
2. **Entrega menos texto que Fixed Size con un resultado similar:** 1 894 frente a 2 394 tokens en los 5 primeros chunks, un 21 % menos.
3. **No necesita ningún modelo para dividir el texto.** Corta por la numeración de los títulos, mientras que las estrategias por embeddings o por LLM requieren calcular similitudes o llamar a un modelo por cada documento.
4. **Cada chunk es una sección real del documento.** La cita que acompaña a la respuesta señala una sección concreta, mientras que Fixed Size corta cada 512 tokens sin mirar el contenido y puede separar los pasos de un mismo procedimiento.

Dense X obtiene el mejor Acc@1 (0,723) y el mejor MRR@10 (0,809): cuando encuentra la respuesta, la proposición correcta suele quedar primera. Sin embargo, su Recall@10 es de solo 0,350 y su nDCG@10 de 0,494, porque la información de una evidencia queda repartida en muchas proposiciones de unos 32 tokens que no caben entre los 10 primeros resultados. Además, el generador recibe apenas 244 tokens, sin el contexto que rodea a cada proposición.

### 5.2. LLM generador

Se compararon diez modelos abiertos que caben en una GPU L40S de 48 GB: tres de 8–12 B y siete de la clase de ∼30 B, en FP8 (Kimi en 4 bits, porque en FP8 no cabe). Cuatro se eligieron por su baja alucinación en el Vectara Hallucination Leaderboard [13]; Qwen3.5-9B, el generador anterior, se incluyó como referencia, y los otros cinco son el modelo más reciente de la clase de ∼30 B de cada familia. La Tabla 5 muestra los resultados con el juez gpt-oss-120b sobre las 170 preguntas verificadas.

- **Alucinación.** Qwen3.8-27B tiene la menor tasa (11,2 %, 19 de 170 respuestas) y la mayor *faithfulness* (0,956). Le siguen GLM-4.7-Flash, Gemma 4 31B, Gemma 4 26B-A4B y Qwen3-8B, entre 14,1 % y 15,3 % (24 a 26 respuestas): una diferencia de 5 a 7 preguntas sobre 170.
- **Rechazo cuando no hay información.** Gemma 4 26B-A4B y Gemma 4 31B tienen el mayor rechazo correcto (97,6 %). Los modelos que menos se abstienen (Qwen3-8B, Gemma 3 12B, Granite 4.0-H y Kimi, entre 75,3 % y 85,3 %) son también los que más inventan cuando los chunks no contienen la respuesta (de 8,8 % a 21,2 %).

**Tabla 5.** LLM generadores (juez gpt-oss-120b, 170 preguntas verificadas por el autor, mismos chunks para todos los modelos). Alucinación y abstención falsa sobre las 170 respondibles; *faithfulness*, exactitud y completitud sobre las respondibles en que el modelo respondió; rechazo correcto e inventa sin información sobre las 170 no respondibles. Latencia: mediana de 20 preguntas con un solo usuario.

| Modelo | Alucinación (%) | *Faithf.* | Rechazo correcto (%) | Inventa sin información (%) | Abst. falsa (%) | Exactitud | Completitud | Latencia (s) |
|---|---|---|---|---|---|---|---|---|
| Qwen3.8-27B | **11,2** | **0,956** | 97,1 | 1,2 | 0,6 | 0,824 | 0,759 | 3,76 |
| GLM-4.7-Flash | 14,1 | 0,919 | 91,8 | 4,7 | 21,8 | 0,816 | 0,698 | 0,90 |
| Gemma 4 31B | 14,7 | 0,933 | **97,6** | **0,6** | 4,7 | **0,829** | 0,796 | 4,01 |
| **Gemma 4 26B-A4B** | 15,3 | 0,940 | **97,6** | 1,2 | 2,9 | 0,824 | **0,837** | 0,82 |
| Qwen3-8B | 15,3 | 0,941 | 85,3 | 12,4 | **0,0** | 0,784 | 0,737 | 1,90 |
| Qwen3.5-9B | 20,0 | 0,902 | 95,9 | 2,9 | 0,6 | 0,794 | 0,785 | 1,68 |
| Gemma 3 12B | 22,4 | 0,910 | 84,7 | 8,8 | 0,6 | 0,802 | 0,751 | 2,42 |
| Granite 4.2 30B | 23,5 | 0,911 | 94,7 | 3,5 | 0,6 | 0,799 | 0,737 | 4,74 |
| Granite 4.0-H Small | 25,9 | 0,859 | 77,6 | 20,0 | **0,0** | 0,757 | 0,719 | 1,34 |
| Kimi-Linear-48B (4 bits) | 31,8 | 0,870 | 75,3 | 21,2 | 1,2 | 0,812 | 0,783 | **0,72** |

- **Abstención falsa.** GLM-4.7-Flash alucina poco, pero se abstiene en el 21,8 % de las preguntas que sí tienen respuesta.
- **Latencia.** Kimi es el más rápido (0,72 s), pero también el que más alucina (31,8 %). Gemma 4 26B-A4B es el segundo (0,82 s): 4,6 veces más rápido que Qwen3.8-27B (3,76 s) y 4,9 veces más rápido que Gemma 4 31B (4,01 s), con el que empata en rechazo y del que lo separa una pregunta en alucinación (15,3 % frente a 14,7 %). Su velocidad viene de la arquitectura MoE: calcula con ∼3,8 B de sus 25,8 B parámetros por token.
- **Tamaño.** Un modelo mayor no garantiza menos alucinación: Qwen3-8B (15,3 %) empata con Gemma 4 26B-A4B y alucina menos que Granite 4.2 30B (23,5 %), Granite 4.0-H Small (25,9 %) y Kimi (31,8 %).

Por ese equilibrio entre alucinación, rechazo correcto y latencia se eligió Gemma 4 26B-A4B como generador. Frente a Qwen3.5-9B, el generador anterior, alucina en 26 respuestas en lugar de 34 (15,3 % frente a 20,0 %).

## 6. Conclusiones

Se construyó un asistente RAG sobre 72 documentos internos usando solo modelos abiertos en servidores propios, y se evaluaron por separado la estrategia de *chunking* y el LLM generador, con las mismas preguntas verificadas y un único juez.

1. **La sección jerárquica es la mejor estrategia de *chunking* para estos documentos.** Obtiene el mayor Recall@10 (0,959) y el mayor nDCG@10 (0,796) entre nueve estrategias, entrega un 21 % menos de texto que Fixed Size y no necesita ningún modelo para dividir el texto.
2. **Gemma 4 26B-A4B ofrece el mejor equilibrio como generador.** Qwen3.8-27B alucina menos (11,2 % frente a 15,3 %, siete preguntas de diferencia), pero tarda 3,76 s por respuesta frente a 0,82 s. Gemma 4 26B-A4B es el segundo más rápido, el tercero en *faithfulness* (0,940), está empatado en el cuarto lugar en alucinación con Qwen3-8B y tiene el mayor rechazo correcto (97,6 %).

3. **El tamaño no garantiza menos alucinación:** Qwen3-8B alucina igual que Gemma 4 26B-A4B y menos que tres modelos de ∼30 B.

*Limitaciones.* Las preguntas las escribió un LLM a partir del mismo texto que se indexa, por lo que usan el vocabulario de los documentos y pueden ser más fáciles de recuperar que las de usuarios reales. Las respuestas las califica un único LLM juez (gpt-oss-120b), que marca como no respaldadas algunas afirmaciones correctas, como la atribución "según el procedimiento X"; por eso las tasas de alucinación deben leerse en términos relativos. La latencia se midió con un solo usuario, sin consultas simultáneas. Como trabajo futuro se propone evaluar con preguntas reales de usuarios, medir la latencia con varios usuarios a la vez y revisar a mano una muestra de los veredictos del juez.

**Agradecimientos.** Se agradece a la empresa. por facilitar los documentos institucionales y el servidor que permitieron realizar las pruebas de este trabajo.

## Referencias

1. Chen, J., Lin, H., Han, X., Sun, L.: Benchmarking large language models in retrieval-augmented generation. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 17754–17762 (2024)
2. Chowdhury, T.: Retrieval-augmented generation (RAG) for large language models: A comprehensive survey. International Journal on Engineering Artificial Intelligence Management, Decision Support, and Policies **3**(1), 9–21 (2026). https://doi.org/10.63503/j.ijaimd.2026.233
3. Chunking-Research: código de evaluación de métodos de *chunking*, https://github.com/ApriiM/Chunking-Research, last accessed 2026/10/23
4. Cormack, G.V., Clarke, C.L.A., Büttcher, S.: Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: Proceedings of the 32nd International ACM SIGIR Conference. pp. 758–759 (2009)
5. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems. vol. 33, pp. 9459–9474 (2020)
6. Ma, X., Gong, Y., He, P., Zhao, H., Duan, N.: Query rewriting in retrieval-augmented large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 5303–5315 (2023)
7. MTEB leaderboard, https://huggingface.co/spaces/mteb/leaderboard, last accessed 2026/10/23
8. OpenAI: gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925 (2025)
9. Ouyang, L., et al.: OmniDocBench: Benchmarking diverse PDF document parsing with comprehensive annotations. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2025), https://github.com/opendatalab/OmniDocBench
10. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval **3**(4), 333–389 (2009)
11. Śmigielski, M., Rajkowski, M., Zbrocki, M., Bernacki-Janson, M., Kunicki, K., Godziszewska, J., Piasecki, M., Wojtasik, K.: Chunking methods on retrieval-augmented generation – effectiveness evaluation against computational cost and limitations. In: 30th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2026), Procedia Computer Science (2026), https://arxiv.org/abs/2606.00881

12. TeleOCR, https://github.com/caipeng328/TeleOCR, last accessed 2026/10/21
13. Vectara: Hallucination leaderboard, https://huggingface.co/spaces/vectara/leaderboard, last accessed 2026/10/23